

Full-Stack Developer Challenge

Build a Mini Shop

Mobile app · REST API · Admin dashboard — one connected system.

Expo React Native

Node.js Fastify

Supabase

TypeScript + Tailwind

Time expectation: 8–12 hours · TypeScript throughout the entire codebase.
Submission: GitHub link + MP4 screen recording · details on the last page.

Tech Stack

Layer	Technology
Mobile App	Expo SDK (latest stable) · React Native · TypeScript
Backend API	Node.js · Fastify · TypeScript
Database / Auth	Supabase (PostgreSQL + Supabase Auth + Storage)
Web Dashboard	React (Vite) · TypeScript · Tailwind CSS

Part 1 — Mobile App

Authentication

- Login — email & password
- Registration — name, email, password
- Forgot Password → Supabase password reset email
- JWT stored in Expo SecureStore · auto-logout on expiry

Product Catalogue

- Grid / list of products: image, name, price
- Search bar + category filter tabs
- Pull-to-refresh · loading skeletons · empty state

Cart & Checkout

- Cart screen: items, quantity controls, subtotal
- Checkout → creates order via API → confirmation screen

Order History

- List of the user's past orders with status badge
- Tap to see order items detail

Navigation & Design

- Bottom tab bar: Shop · Cart · Orders · Profile
- **A good eye for design and UI/UX is a prerequisite — not a bonus. The app must look and feel polished.**
- Consistent colors, typography, spacing — no tutorial-style layouts
- Loading, error, and empty states on every screen

Part 2 — Backend API

Auth

Method	Route	Description
POST	/auth/register	Register — name, email, password
POST	/auth/login	Login → returns JWT
POST	/auth/forgot-password	Trigger Supabase password reset email
GET	/auth/me	Current user profile (protected)

Products

Method	Route	Description
GET	/products	List active products — supports search & category filter
GET	/products/:id	Single product detail
POST	/products	Create product — admin only
PATCH	/products/:id	Update product — admin only
DELETE	/products/:id	Soft-delete product — admin only

Orders

Method	Route	Description
POST	/orders	Place order (authenticated user)

GET	/orders/my	Current user's order history
GET	/orders	All orders — paginated, admin only
PATCH	/orders/:id/status	Update order status — admin only

Requirements

- Zod validation on every route
- Consistent error format: { statusCode, error, message }
- Role-based access: customer vs. admin (Supabase RLS + JWT role claim)
- Product images stored in Supabase Storage
- .env for all secrets — no hardcoded values

Database (Supabase / PostgreSQL)

- profiles — extends Supabase Auth: name, role
- categories — id, name, slug
- products — id, name, description, price, image_url, category_id, is_active
- orders — id, user_id, status, total_amount, created_at
- order_items — order_id, product_id, quantity, unit_price

Part 3 — Admin Web Dashboard

Pages

- Login — Supabase Auth, admin accounts only
- Dashboard — KPI cards: orders today, revenue, active products
- Products — table with create / edit / toggle active, image upload
- Orders — table with status filter, update status, order detail view

Design

- Sidebar navigation · responsive (desktop & tablet)
- Loading skeletons & toast notifications on all actions
- Clean professional look — Tailwind utility classes throughout

Bonus Points

- Real-time order updates via Supabase Realtime on the mobile app
- Dark / Light theme (mobile & dashboard)
- Micro-animations with Reanimated 3

Deliverables

1 — Source Code (GitHub or ZIP)

- Three sub-projects: /mobile /backend /dashboard
- README.md in each sub-project: setup instructions + .env.example
- Seed data: at least 10 products across 3 categories
- Test credentials: one customer account + one admin account

2 — Video Recording (Required)

Record a screen recording of your finished application and explain it in English.

Duration: 4–5 minutes

Language: English

Format: MP4 — upload to Google Drive, Loom, or WeTransfer
Include the link in your repository README.

What to cover in the video:

1. Walk through the mobile app: register, browse products, add to cart, place an order.
2. Open the admin dashboard: find the order just placed, update its status to 'Processing'.
3. Briefly show your code structure and explain one technical decision you are proud of.

Evaluation Criteria

Criterion	Weight	What we look at
Functionality	35 %	All features work end-to-end as specified
Code Quality	30 %	Clean, typed, structured — no shortcuts
UI / UX Design	20 %	Polished mobile + dashboard; good UX flow
Security	10 %	Auth, RLS, input validation, no leaks
Bonus	+5 %	Outstanding extra additions

Questions? Do not hesitate to reach out — we look forward to seeing your work. Good luck!